

SENAI FATESG

Análise e Desenvolvimento de Sistemas — 3º Semestre
Arquitetura e Projeto de Software

ADR-002

BaseModel e Spring Data JPA

Status: **Em vigor** **Data:** Aula 02 **Autor:** Equipe

Caio Abreu | Cassiano Abreu | Gabriel Naoki | Wyllian Mariano
SENAI FATESG — Goiânia, GO — 01 de abril de 2025

Contexto

Todas as entidades do sistema precisavam de campos comuns de auditoria: identificador único, data de criação e indicador de status ativo/inativo. Sem uma estratégia centralizada, esses campos seriam duplicados manualmente em cada entidade, gerando retrabalho, inconsistências e violando o princípio DRY (Don't Repeat Yourself). Além disso, era necessário definir como o acesso ao banco de dados seria abstraído para evitar SQL manual disperso pelo código.

Decisão

Criar uma classe abstrata `BaseModel` anotada com `@MappedSuperclass` contendo os campos comuns a todas as entidades: `id` (UUID gerado automaticamente), `ativo` (boolean para soft delete) e `dataHoraCriacao` (Date para auditoria). Criar também uma classe `BaseDTO` espelhando os campos `id` e `active` para o contrato da API. Adotar `JpaRepository` do Spring Data como base para todos os repositórios, abstraindo o SQL manual.

Estrutura do `BaseModel`:

```
@MappedSuperclass
public abstract class BaseModel {
    @Id @GeneratedValue
    private UUID id;
    private boolean ativo;
    private Date dataHoraCriacao;
}
```

Justificativa

- `BaseModel` elimina duplicação de campos de auditoria — princípio DRY aplicado na camada de entidades
- UUID como chave primária evita colisões em ambientes distribuídos e é mais seguro que IDs sequenciais expostos na API
- O campo `ativo` viabiliza soft delete — o registro nunca é removido fisicamente, preservando histórico e integridade referencial
- `JpaRepository` abstrai completamente o SQL manual, permitindo foco total na lógica de negócio
- Spring Data gera automaticamente queries por convenção de nomes (ex: `findById`, `existsByCpf`, `findByAtivo`)
- `BaseDTO` desacopla o contrato da API da estrutura interna do banco de dados

Alternativas Consideradas

- Herança com `SINGLE_TABLE` (`@Inheritance`) — descartado por misturar dados de domínios distintos em uma única tabela
- JDBC puro sem JPA — descartado pela verbosidade e necessidade de SQL manual para operações simples
- IDs sequenciais (`Long auto_increment`) — descartado em favor de UUID para maior segurança e escalabilidade

- Campos de auditoria duplicados em cada entidade — descartado por violar DRY e gerar retrabalho

Consequências

- Todas as entidades herdam automaticamente id, ativo e dataHoraCriacao sem nenhuma repetição de código
- Soft delete implementado via campo ativo — exclusões são lógicas, não físicas, preservando histórico
- A dependência do Spring Data JPA impõe uso obrigatório de uma implementação JPA (Hibernate por padrão)
- Migrações de schema precisam ser gerenciadas manualmente ou via ferramenta (ex: Flyway) em versões futuras
- Qualquer novo domínio herda automaticamente o padrão de auditoria sem esforço adicional